

Challenge 1 – Monte Carlo

8 hardware threads:

Threads	Time (s)	Speedup	Efficiency
1	1.0037	1.00×	100.0%
2	0.5146	1.95×	97.5%
4	0.2745	3.66×	91.4%
8	0.2161	4.64×	58.0%

A. Baseline and maximum runtime

Baseline, 1 thread:

$T_1 = 1.0037$ seconds

Maximum-thread runtime, 8 threads:

$T_8 = 0.2161$ seconds

The resulting speedup was:

4.64×

and efficiency was:

58.0%

B. Why is the warm-up required?

The warm-up call is required because Numba uses JIT (Just-In-Time) compilation. The first execution compiles the Python function into optimized machine code, which introduces compilation overhead. If the first call were included in the benchmark, the measured execution time would include compilation time rather than representing the actual runtime of the parallel computation. The warm-up therefore ensures that the benchmark measures the computation itself.

The lab explicitly identifies the warm-up as excluding JIT compilation overhead from the benchmark.

C. Why does efficiency decrease?

results demonstrate this clearly:

1 thread → 100.0%
2 threads → 97.5%
4 threads → 91.4%
8 threads → 58.0%

Parallel efficiency decreased as the number of threads increased. With more threads, the processor cannot continue scaling perfectly because the threads compete for shared hardware resources. Memory-system contention, cache effects, thread scheduling overhead, and the distinction between physical CPU resources and simultaneous hardware threads can limit scaling. Therefore, doubling the number of threads does not necessarily halve the execution time.

D. Why doesn't `inside_circle += 1` corrupt the result?

The increment is handled as a reduction operation by Numba's parallelization mechanism. Each parallel iteration contributes to the overall `inside_circle` result, and the partial results are combined rather than all threads directly modifying one shared counter simultaneously. This prevents a race condition in the reduction.

Challenge 2 – Mandelbrot

Decomposition	Time
Rows	1.645 s
Columns	1.605 s

A. Row vs column

The row-parallel implementation took **1.645 seconds**, while the column-parallel implementation took **1.605 seconds**. Therefore, the column-parallel implementation completed approximately **0.040 seconds faster** on my machine. The difference was relatively small.

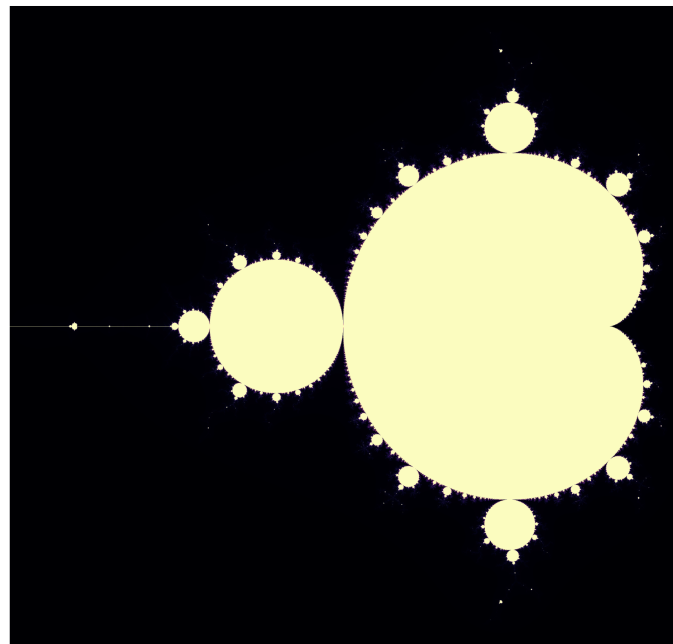
B. Cache lines

NumPy uses a C-contiguous, row-major memory layout. Values belonging to the same row are stored next to each other in memory. Accessing neighboring memory locations allows the CPU to make better use of cache lines. Changing the parallelization axis changes the pattern in which memory is accessed, which can influence cache behavior and execution time.

C. Load imbalance

Mandelbrot calculations have an uneven workload because different points require different numbers of iterations. Points outside the set can escape after only a few iterations, while points inside the set may continue until the maximum iteration count. If one thread receives computationally expensive rows while another receives easier rows, the easier thread can finish earlier and remain idle. Dynamic scheduling can redistribute unfinished work among available threads, improving utilization.

Mandelbrot 2500x2500 (Render: 1.64s)



D. Output

The Mandelbrot image was successfully generated as `mandelbrot_output.png`. The file size was approximately **468 KB**

Challenge 3 – Heat diffusion

Precision	Time	Throughput
float64	0.152 s	4443.00 Mcells/s
float32	0.077 s	8821.35 Mcells/s

A. Benchmark

With float64, the heat-diffusion simulation took **0.152 seconds** and achieved **4443.00 Megacells/sec**. With float32, it took **0.077 seconds** and achieved **8821.35 Megacells/sec**.

B. Why did float32 improve performance?

The runtime decreased from 0.152 seconds to 0.077 seconds, giving an improvement factor of approximately **1.97×**. A float32 value requires half the storage of a float64 value. Since the stencil repeatedly reads and writes large arrays and is memory-bandwidth bound, reducing the amount of data transferred through the memory subsystem can substantially improve throughput.

C. Why doesn't twice as many CPU cores give 2× speedup?

The stencil computation is limited by memory bandwidth rather than purely by CPU computation. As more CPU cores are added, they increasingly compete for access to the same memory subsystem. Once the available memory bandwidth is saturated, additional cores cannot obtain data any faster. Consequently, doubling the number of CPU cores does not necessarily produce a 2× performance improvement.

Final Table 1

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1	120,000,000	1.0037	1.00×	100.0%
Challenge 1: Monte Carlo	2	120,000,000	0.5146	1.95×	97.5%
Challenge 1: Monte Carlo	4	120,000,000	0.2745	3.66×	91.4%
Challenge 1: Monte Carlo	8	120,000,000	0.2161	4.64×	58.0%
Challenge 2: Mandelbrot (Rows)	8	2500 × 2500	1.645	N/A	N/A
Challenge 2: Mandelbrot (Cols)	8	2500 × 2500	1.605	N/A	N/A
Challenge 3: Heat Stencil	8	1500 × 1500 × 300	0.152	N/A	N/A